Performance, Safety, and Engineering Trade-offs of Rust and C++ in RAN Control-Plane Systems

A Tail-Latency, Safety, and Migration Study of a
UE Context Management Subsystem

Ha Vien
University of Helsinki Department of Computer Science

Thesis Plan — June 16, 2026

Abstract

Modern Radio Access Network (RAN) control-plane software is predominantly written in C++, a language that delivers high performance but carries well-known memory-safety and concurrency risks. Rust promises compile-time memory and thread safety with comparable performance, yet its suitability for telecom-grade, latency-critical control-plane systems is not well understood. This thesis develops two functionally equivalent implementations of a representative RAN subsystem — a UE (User Equipment) Context Management service — in modern C++ and in Rust, and compares them under a realistic synthetic signaling workload. Rather than re-confirming the well-established result that Rust and C++ achieve similar average throughput, the study focuses on three decision-useful and under-explored dimensions: (i) *tail latency and jitter* under overload, (ii) the classes of defects that Rust prevents at compile time versus those C++ catches only at runtime (via controlled fault injection), and (iii) the practical *cost of incremental adoption* of Rust within an existing C++ codebase. The expected contribution is a reproducible, telecom-relevant benchmarking framework and a set of evidence-based engineering guidelines for language selection in cloud-native RAN evolution.

Contents

1 Background and Motivation **2**

2 Problem Statement **2**

3 Research Questions **2**

4 Scope of the Work **3**

5 Methodology **3**
5.1 Shared Specification (fairness baseline) 3
5.2 System Design 3
5.3 Workload Generator 4
5.4 Experimental Setup 4
5.5 Metrics 4
5.6 Validity and Fairness Controls 4

6 Evaluation Plan and Hypotheses **5**

7 Expected Contributions **5**

8 Limitations and Threats to Validity **5**

9 Tools and Technologies **6**

10 Timeline **6**

---

1 Background and Motivation

Modern RAN control-plane systems (e.g. the L3 components of a gNB) are primarily implemented in C++. These systems must satisfy demanding non-functional requirements:

- very high event rates (UE mobility, signaling storms);
- strict and predictable latency bounds;
- long-lived software with continuously evolving complexity;
- heavily concurrent workloads.

At the same time, C++ introduces long-standing engineering challenges:

- memory-safety defects (use-after-free, buffer overflow, double-free);
- concurrency defects (data races, deadlocks, iterator invalidation);
- high maintenance cost and review burden in large codebases.

Rust has emerged as a systems language offering memory safety and data-race freedom *guaranteed at compile time* by its ownership and borrowing model, a strong concurrency story, and modern tooling (Cargo, integrated testing, first-class sanitizer-free safety). However, the consensus that "Rust performs roughly like C++" has been established mostly on general-purpose benchmarks. Whether those properties hold — and what they cost — for *stateful, concurrent, latency-critical* telecom control-plane workloads remains unclear. This thesis evaluates whether Rust can be a viable alternative or complement to C++ in RAN control-plane development.

2 Problem Statement

For telecom control-plane workloads it is currently unclear:

- whether Rust matches C++ not only in average throughput but in *tail latency and predictability*, which dominate RAN deadline behaviour;
- which concrete classes of memory and concurrency defects present in C++ are *prevented at compile time* by Rust, and which still require runtime tooling in C++;
- what overhead, if any, Rust's safety model imposes under high-throughput, high-concurrency load;
- what the practical cost and risk are of adopting Rust *incrementally* inside an existing, large C++ RAN codebase.

3 Research Questions

The research questions are ordered to lead with the most novel, decision-useful dimensions (predictability and migration) rather than raw average performance.

RQ1 How do Rust and modern C++ compare in *latency predictability* (mean, p99, p99.9 tail latency, and jitter) for a RAN control-plane workload under increasing load and overload?

RQ2 Which classes of memory-safety and concurrency defects that occur in C++ are eliminated at compile time in the Rust implementation, and at what stage (compile time vs runtime) and cost are the remaining ones detected?

RQ3 What is the overhead, if any, of Rust's safety model (bounds checks, ownership-driven copies/clones, abstraction cost) on throughput, CPU utilisation, and memory behaviour?

RQ4 What are the practical implications — effort, interoperability (FFI), and risk — of adopting Rust incrementally within an existing C++ RAN software ecosystem?

4 Scope of the Work

To keep the work feasible, the thesis focuses on a single representative subsystem of the gNB control plane: a **UE Context Management** service. This subsystem is *modeled on*, but deliberately simplified from, a real gNB-CU-CP UE service domain, which owns UE context lifecycle, RRC procedure execution, and mobility/handover. It is a good choice because it is:

- core RAN L3 functionality;
- performance- and latency-sensitive;
- inherently stateful and concurrent;
- self-contained enough to specify precisely and benchmark fairly.

The modeled functionality includes:

- UE context creation and deletion;
- state transitions (e.g. idle $\leftrightarrow$ connected);
- signaling message handling;
- concurrent context lookup under load.

Explicitly out of scope: the full gNB system, PHY/L2 processing, real traffic traces, and production deployment/operational constraints (see Section 8).

5 Methodology

5.1 Shared Specification (fairness baseline)

Before any code is written, a single language-agnostic specification is frozen: the public API, the UE state machine, the data structures and their algorithmic strategy (e.g. sharded hash map keyed by UE identifier), the locking/sharding granularity, and the concurrency model. Both implementations must conform to this specification so that the comparison measures *language* differences rather than *design* or *skill* asymmetry. This shared specification is itself a deliverable.

5.2 System Design

Two functionally equivalent implementations are developed against the shared specification:

- **C++ implementation:** modern C++ (C++17/20), STL, RAII, standard threading.
- **Rust implementation:** ownership-based memory model, safe concurrency.

A single concurrency model is chosen for both (a shared-nothing / sharded worker-thread model, which mirrors how real RAN L3 partitions UE state) to keep the two systems comparable. Async-vs-threads mixing is intentionally avoided.

5.3 Workload Generator

A synthetic but *spec-justified* telecom workload is generated. The event mix and rates are anchored to published 3GPP/RAN load characteristics and the real subsystem's procedure set rather than chosen arbitrarily:

- UE creation bursts (arrival process);
- mobility / handover events;
- context updates and state transitions;
- concurrent signaling events.

Control parameters: target event rate (events/sec) and number of active UEs (scale tests), swept from nominal load through saturation/overload.

5.4 Experimental Setup

- Linux-based system; identical hardware for both implementations.
- CPU pinning; frequency scaling / turbo disabled; warm-up runs discarded.
- Controlled compiler optimisation (`-O2`/`-O3`; Rust `release` with comparable codegen settings).
- Identical memory allocator on both sides (e.g. jemalloc) to isolate language effects from allocator effects.
- Many repeated runs; variance and confidence intervals reported.
- Full configuration and scripts published for reproducibility.

5.5 Metrics

Category	**Metrics**
Latency predictability	per-event latency: mean, median, **p99 / p99.9 / p99.99**, max; jitter; behaviour at and beyond saturation
Throughput	sustained events/sec; throughput at fixed latency-SLO
Resource use	CPU utilisation; memory usage over time; allocation rate/patterns
Concurrency robustness	race-condition susceptibility; stability and graceful degradation under stress/overload
Safety	defect classes prevented at compile time vs runtime (fault-injection study, §6)
Software quality	LOC, cyclomatic complexity, count and scope of Rust `unsafe` blocks; qualitative error-proneness

5.6 Validity and Fairness Controls

The central threat to this study is implementation bias. It is mitigated by: the frozen shared specification (§5.1); identical algorithms, data structures, and concurrency model; identical allocator and build optimisation; independent functional *parity testing* (both implementations must pass the same conformance test suite); and, where possible, review of both implementations for idiomatic quality. All deviations are documented as threats to validity (Section 8).

6 Evaluation Plan and Hypotheses

Results are framed as *hypotheses to test*, not foregone conclusions.

Performance (RQ1, RQ3). Hypothesis: Rust and C++ achieve similar mean throughput, but they may differ measurably in *tail latency and jitter* under overload; any safety-model overhead in Rust is expected to be small in realistic workloads. The study reports where (if anywhere) each language wins and by how much.

Safety via fault injection (RQ2). Rather than asserting that Rust "has no bugs," a catalogue of classic C++ defects is deliberately injected into the C++ implementation (e.g. use-after-free, data race, double-free, iterator invalidation, unguarded shared mutation). For each defect we record: whether the analogous construct is *rejected at compile time* by Rust's borrow checker / type system; whether C++ detects it only at runtime via sanitizers (ASan/TSan) and at what performance cost; and whether it can escape detection entirely. The output is a defect-class taxonomy mapping {compile-time-prevented in Rust / lint-preventable / runtime-only / undetected}. Optionally, the taxonomy is cross-checked against the real defect classes observed in a production telecom codebase's bug history.

Migration (RQ4). A qualitative-plus-quantitative assessment of incremental adoption: building a small Rust module that interoperates with the C++ implementation via FFI, measuring the interop boilerplate, the safety boundary at the FFI seam, and the engineering effort. The deliverable is a set of concrete guidelines for when and how Rust adoption is worthwhile in an existing C++ RAN ecosystem.

7 Expected Contributions

1. A reproducible, telecom-relevant benchmarking framework for control-plane workloads (workload generator + harness + analysis scripts).
2. A frozen shared specification and two parity-tested implementations of a RAN control-plane subsystem in C++ and Rust.
3. A quantitative comparison emphasising **tail latency and predictability**, not just average throughput.
4. A **defect-class taxonomy** (via fault injection) showing exactly which C++ failure modes Rust prevents at compile time and at what cost the rest are caught — the strongest novel contribution.
5. Evidence-based engineering and **migration guidelines** (including FFI interop cost) for language selection in cloud-native RAN evolution.

8 Limitations and Threats to Validity

Internal Implementation bias between the two versions (mitigated by §5.6); single-author skill asymmetry.

External A synthetic workload (not real captured traces); a single simplified subsystem rather than full gNB complexity; results may not generalise to PHY/L2 or to other hardware.

Construct Whether the chosen metrics (esp. tail latency, code complexity proxies) faithfully capture "predictability" and "error-proneness."

Scope Control-plane only (not PHY/L2); excludes full production deployment and operational constraints.

9 Tools and Technologies

- Languages: modern C++ (C++17/20); Rust (stable toolchain).
- Build: CMake (C++); Cargo (Rust).
- Benchmarking: Google Benchmark / custom harness (C++); Criterion / `hyperfine` (Rust); shared latency-histogram tooling (e.g. HdrHistogram).
- Profiling & safety: `perf`, Valgrind, AddressSanitizer (ASan), ThreadSanitizer (TSan).
- Allocator: common allocator (e.g. jemalloc) on both sides.
- Environment: Linux; pinned CPUs; controlled clocking.

10 Timeline

One month, one job. The plan runs July through November — five months, five simple goals — with submission **before December**.

Month	One-word goal	What happens
July	**Plan**	Literature review; freeze the spec and architecture; design the workload generator and metrics.
August	**Build C++**	Implement the C++ subsystem and the benchmarking harness.
September	**Build Rust**	Implement the Rust subsystem; pass parity tests. *(Safety-net checkpoint: both versions work.)*
October	**Measure**	Full benchmarking, fault-injection, and FFI/migration study; collect all data.
November	**Write**	Analyse results, write the thesis, and submit.

The five-word memory hook: *Plan → Build C++ → Build Rust → Measure → Write.* The single rule: if September ends with both versions running, the thesis is safe.